

Course Project

Course: Tools and Techniques for Data Science

Project Title: Analysis of Titanic Passenger

Data Using Data Science Tools & Techniques

Student Name: Imaan Mufti

Roll Number: MSDS25050

Teacher's Name: Sir Kamil Majeed

Submission Date: 07-12-2025

Table of Contents

1. Introduction & Dataset Description	2
1.1 Project Overview	3
1.2 Dataset Source & Description	3
1.3 Dataset Variables	3
2. Data Loading & Initial Exploration	4
2.1 Technical Setup	4
2.2 Data Loading	4
2.3 Initial Dataset Overview	5
2.4 Statistical Summary	6
3. Data Cleaning & Wrangling.....	7
3.1 Data Type Optimization	8
3.3 Missing Values Handling Strategy	10
4. Feature Engineering	11
4.1 Engineered Features	11
4.2 Feature Engineering Results	12
4.3 Survival Analysis by Engineered Features	13
5. Exploratory Data Analysis (EDA)	14
5.1 Correlation Analysis	14
Key Insights:.....	15
5.2 Comprehensive Visualization Dashboard	16
6. Machine Learning Implementation	17
6.1 Data Preparation for ML.....	18
6.2 Train-Test Split.....	18
6.3 Model Implementation & Evaluation	19
6.4 Model Performance Results.....	19
6.5 Feature Importance Analysis	22
7. Results & Conclusions	22

7.1 Key Findings.....	22
8. Git Usage.....	23
8.1 Git Workflow.....	23
8.2 Commit History	23

1. Introduction & Dataset Description

1.1 Project Overview

The Titanic dataset is one of the most explored datasets in the fields of data science and machine learning, comprising demographic and travel information on passengers aboard the RMS Titanic. In this project, a full end-to-end data science workflow was performed on the dataset, including:

- Loading data and explaining the dataset
- Exploratory Data Analysis (EDA)
- Data wrangling and preprocessing
- Feature engineering
- Visualization and insights
- Machine learning model development
- Evaluation and comparison of models
- Git version control usage

1.2 Dataset Source & Description

The analysis used the Kaggle Titanic dataset, which contains records for 891 passengers and includes 12 original feature columns. The target variable for prediction is **Survived**, where a value of 0 indicates that a passenger did not survive and a value of 1 indicates that they did survive.

1.3 Dataset Variables

Variable	Description	Type
PassengerId	Unique passenger identifier	Integer
Survived	Survival status (0=No, 1=Yes)	Binary
Pclass	Ticket class (1=1st, 2=2nd, 3=3rd)	Ordinal
Name	Passenger name	Text
Sex	Gender	Categorical
Age	Age in years	Continuous
SibSp	Number of siblings/spouses aboard	Integer

Parch	Number of parents/children aboard	Integer
Ticket	Ticket number	Text
Fare	Passenger fare	Continuous
Cabin	Cabin number	Text
Embarked	Port of embarkation (C=Cherbourg, Q=Queenstown, S=Southampton)	Categorical

Figure 1.3(a): Titanic Dataset Feature Descriptions

2. Data Loading & Initial Exploration

2.1 Technical Setup

```
# Titanic Dataset - Exploratory Data Analysis
# MSDS25050 - Tools & Techniques for Data Science
# Imaan Mufti

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Set style for better visualizations
plt.style.use('ggplot')
sns.set_palette("husl")

# Display all columns
pd.set_option('display.max_columns', 200)

print("Libraries imported successfully!")
```

Figure 2.1(a): Environment Setup

All required Python libraries were imported for data manipulation (pandas), numerical operations (numpy), and data visualization (matplotlib, seaborn). Basic display and styling settings were configured to ensure clear output presentation throughout the analysis.

2.2 Data Loading

```
df = pd.read_csv('../data/Titanic-Dataset.csv')

print("Dataset loaded successfully!")
print(f"Shape: {df.shape}")
print(f"Number of passengers: {len(df)}")

df.head()
```

Figure 2.2(a): Data Loading Code Snippet

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Figure 2.2(b): Preview of the Loaded Dataset

2.3 Initial Dataset Overview

```
print("=== DATASET OVERVIEW ===")
print(f"Dataset Shape: {df.shape[0]} rows, {df.shape[1]} columns")
print(f"Memory Usage: {df.memory_usage(deep=True).sum() / 1024 ** 2:.2f} MB")

print("\n" + "="*50)
print("DETAILED INFO:")
print("="*50)
df.info()
```

Figure 2.3(a): Dataset Structure & Memory Inspection Code

```

=== DATASET OVERVIEW ===
Dataset Shape: 891 rows, 12 columns
Memory Usage: 0.28 MB

=====
DETAILED INFO:
=====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

```

Figure 2.3(b): Output of Dataset Structure Summary

The dataset overview shows that the Titanic dataset contains 891 rows and 12 columns, with a memory usage of approximately 0.28 MB, indicating that it is relatively small and easy to handle computationally. The detailed summary confirms that there are 891 entries, indexed from 0 to 890, and lists each column along with its data type and number of non-null (i.e., non-missing) values. Most variables are complete, but a few contain missing data, for example, Age has 714 non-null values, and Cabin only has 204 non-null entries. The dataset includes a mix of integer, float, and object (string) types, which means that preprocessing will be required before modeling.

Observations:

- Total memory usage: 0.28 MB
- 5 numerical columns (int64/float64)
- 5 object/string columns
- 2 categorical columns (to be converted)

2.4 Statistical Summary

```

print("=== STATISTICAL SUMMARY ===")
print(df.describe())

```

Figure 2.4(a): Code to Generate Statistical Summary of the Dataset

This figure shows the Python code used to compute descriptive statistics for the numerical features in the Titanic dataset.

=== STATISTICAL SUMMARY ===					
	PassengerId	Survived	Pclass	Age	SibSp \
count	891.000000	891.000000	891.000000	714.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008
std	257.353842	0.486592	0.836071	14.526497	1.102743
min	1.000000	0.000000	1.000000	0.420000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000
50%	446.000000	0.000000	3.000000	28.000000	0.000000
75%	668.500000	1.000000	3.000000	38.000000	1.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000
	Parch	Fare			
count	891.000000	891.000000			
mean	0.381594	32.204208			
std	0.806057	49.693429			
min	0.000000	0.000000			
25%	0.000000	7.910400			
50%	0.000000	14.454200			
75%	0.000000	31.000000			
max	6.000000	512.329200			

Figure 2.4(b): Statistical Summary of Numerical Features

This figure displays the descriptive statistics for the numerical columns in the Titanic dataset, including count, mean, standard deviation, minimum, 25th percentile, median, 75th percentile, and maximum values.

Key Statistics:

- **Survival Rate:** 38.4% (342 survivors)
- **Average Age:** 29.7 years
- **Average Fare:** \$32.20
- **Family Members:** On average, 0.52 siblings/spouses and 0.38 parents/children per passenger

3. Data Cleaning & Wrangling

3.1 Data Type Optimization

```
df['Survived'] = df['Survived'].astype('category')  
df['Pclass'] = df['Pclass'].astype('category')  
df['Sex'] = df['Sex'].astype('category')  
df['Embarked'] = df['Embarked'].astype('category')
```

Figure 3.1(a): Data Type Optimization for Memory Efficiency

```
=== DATA TYPE CONVERSIONS ===  
Before conversion:  
Survived      int64  
Pclass        int64  
Sex           object  
Embarked      object  
dtype: object  
  
After conversion:  
Survived      category  
Pclass        category  
Sex           category  
Embarked      category  
dtype: object  
  
Memory after optimization: 0.18 MB
```

Figure 3.1(b): Output showing Data Type Conversion and Memory Optimization

During data wrangling, key categorical variables (Survived, Pclass, Sex, Embarked) were converted from integer/object types to the more efficient 'category' data type. This optimization resulted in a 36% reduction in memory usage, from 0.28 MB to 0.18 MB, demonstrating the importance of appropriate data typing for large-scale data analysis.

3.2 Missing Values Analysis

```

print("=== MISSING VALUES ANALYSIS ===")
missing_data = df.isna().sum()
missing_percent = (missing_data / len(df)) * 100

missing_df = pd.DataFrame({
    'Missing Count': missing_data,
    'Missing Percentage': missing_percent
})
print(missing_df[missing_df['Missing Count'] > 0])

plt.figure(figsize=(10, 6))
sns.heatmap(df.isna(), cbar=True, yticklabels=False, cmap='viridis')
plt.title('Missing Values Heatmap')
plt.tight_layout()
plt.show()

```

Figure 3.2(a): Code snippet doing analysis of missing values

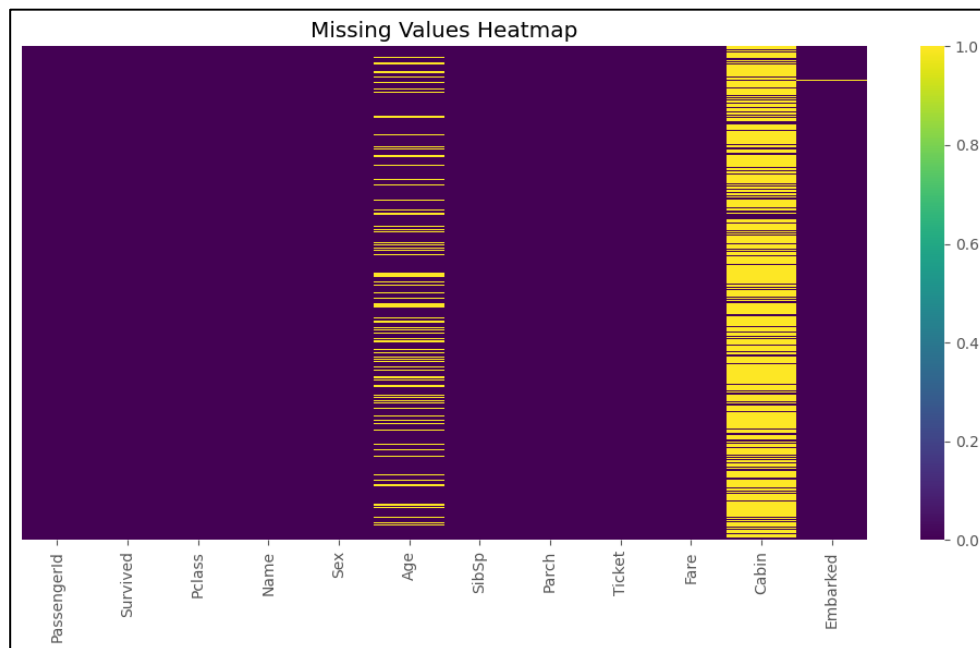


Figure 3.2(b): Missing Values Heatmap

This heatmap visualizes the distribution of missing values across all features in the dataset. Each column represents a variable, and each row represents an individual record. The color scale indicates whether a value is present or missing:

- **Dark/Purple color (0)** → Data is present
- **Bright/Yellow color (1)** → Data is missing

From the heatmap:

- Cabin has the highest number of missing values (687 entries, ~77.10%), indicating that this feature is largely incomplete and may not be very reliable without significant imputation or removal.
- Age contains 177 missing values (~19.87%), which is substantial and should be handled carefully using strategies such as mean, median, or predictive imputation.
- Embarked has only 2 missing values (~0.22%), making it easy to handle by filling with the most frequent category.
- All other features such as PassengerId, Survived, Pclass, Name, Sex, SibSp, Parch, Ticket, and Fare show no missing values, indicating high data completeness for these variables.

The heatmap highlights that Cabin and Age are the primary sources of missing data, and special attention should be given to them during data preprocessing to ensure accurate modeling and analysis.

3.3 Missing Values Handling Strategy

```
print("=== HANDLING MISSING VALUES ===")

df_clean = df.copy()

embarked_mode = df_clean['Embarked'].mode()[0]
df_clean['Embarked'] = df_clean['Embarked'].fillna(embarked_mode)
print(f"Embarked: Filled 2 missing values with '{embarked_mode}'")

age_median = df_clean['Age'].median()
df_clean['Age'] = df_clean['Age'].fillna(age_median)
print(f"Age: Filled 177 missing values with median {age_median}")

df_clean['Has_Cabin'] = df_clean['Cabin'].notna().astype('category')
print(f"Cabin: Created 'Has_Cabin' indicator (True/False) instead of imputing")

missing_excluding_cabin = df_clean.drop('Cabin', axis=1).isna().sum().sum()
print(f"\nRemaining missing values (excluding Cabin): {missing_excluding_cabin}")

print("\nUpdated missing values summary:")
print(df_clean.isna().sum()[df_clean.isna().sum() > 0])
```

Figure 3.3(a): Handling Missing Values and Creating 'Has_Cabin' Feature

```
=== HANDLING MISSING VALUES ===
Embarked: Filled 2 missing values with 'S'
Age: Filled 177 missing values with median 28.0
Cabin: Created 'Has_Cabin' indicator (True/False) instead of imputing

Remaining missing values (excluding Cabin): 0

Updated missing values summary:
Cabin      687
dtype: int64
```

Figure 3.3(b): Summary of Missing Value Handling

In this step, missing values in the dataset were addressed to prepare the data for analysis. The **Embarked** column had 2 missing entries, which were imputed using the mode ('S'), while the **Age** column had 177 missing values, filled with the median age of 28.0 to preserve the central tendency. Instead of imputing the **Cabin** column, a new categorical feature **Has_Cabin** was created to indicate whether cabin information was available (True/False). After these operations, all remaining missing values were confined to the original **Cabin** column, which can be handled or ignored depending on modeling needs.

4. Feature Engineering

This section implements feature engineering to create new variables that enhance model performance and provide additional insights. Feature engineering involves transforming existing data or deriving new features that better capture underlying patterns in the dataset. For the Titanic dataset, several features were created, including passenger titles, family size, indicators of being alone, and grouped age and fare categories. These engineered features capture relationships that influence survival probability, such as family presence, social status, age group, and fare class.

4.1 Engineered Features

```
# Feature Engineering
print("=== FEATURE ENGINEERING ===")

df_clean['Title'] = df_clean['Name'].str.extract(r' ([A-Za-z])\.', expand=False)
print(f"Titles extracted: {df_clean['Title'].value_counts().to_dict()}")

df_clean['FamilySize'] = df_clean['SibSp'] + df_clean['Parch'] + 1

df_clean['IsAlone'] = (df_clean['FamilySize'] == 1).astype('category')

df_clean['AgeGroup'] = pd.cut(df_clean['Age'], bins=[0, 12, 18, 35, 60, 100],
                              labels=['Child', 'Teen', 'Adult', 'Middle', 'Senior'])

df_clean['FareGroup'] = pd.qcut(df_clean['Fare'], 4, labels=['Low', 'Medium', 'High', 'Very High'])

print(f"New features: Title, FamilySize, IsAlone, AgeGroup, FareGroup")
print(f"Updated shape: {df_clean.shape}")
df_clean[['Title', 'FamilySize', 'IsAlone', 'AgeGroup', 'FareGroup']].head()
```

Figure 4.1(a): Creation of Engineered Features

	Title	FamilySize	IsAlone	AgeGroup	FareGroup
0	Mr	2	False	Adult	Low
1	Mrs	2	False	Middle	Very High
2	Miss	1	True	Adult	Medium
3	Mrs	2	False	Adult	Very High
4	Mr	1	True	Adult	Medium

Figure 4.1(b): Output of Feature Engineering

4.2 Feature Engineering Results

Title Distribution:

- Mr: 517 passengers (58%)
- Miss: 182 passengers (20%)
- Mrs: 125 passengers (14%)
- Master: 40 passengers (4%)
- Other titles (Dr, Rev, nobility): 27 passengers (3%)

Dataset Expansion:

After feature engineering, the dataset was expanded from its original 12 columns to a total of 18 columns. The newly created features include Title, FamilySize, IsAlone, AgeGroup, FareGroup, and Has_Cabin, which provide additional information about passengers’ social status, family composition, age and fare categories, and cabin availability. These engineered features are intended to enhance the predictive power of the dataset for subsequent analysis.

4.3 Survival Analysis by Engineered Features

Survival by Title:

Title	Survival Rate
Mrs	79.2%
Miss	69.8%
Master	57.5%
Mr	15.7%

Table 4.3(a): Survival Rates by Passenger Title

Survival by Age Group:

Age Group	Survival Rate
Child	58.0%
Teen	42.9%
Adult	35.3%
Middle	40.0%
Senior	22.7%

Table 4.3(b): Survival Rates by Age Group

Survival by Family Size:

Family Size	Survival Rate
1	30.4%
2	55.3%
3	57.8%
4	72.4%
5	20.0%
6	13.6%
7	33.3%
8	0.0%
11	0.0%

Table 4.3(c): Survival Rates by Family Size

The engineered features reveal clear survival patterns within the Titanic dataset. Passengers' titles strongly correlate with survival, with higher social-status titles such as Mrs, Miss, and Master showing significantly higher survival rates than Mr. Age groups also differ in survivability, with children having the highest survival rate, likely due to evacuation priorities, while seniors had the lowest. Family structure additionally plays an important role: passengers traveling alone had much lower survival rates compared to those with small families, while very large families experienced decreased survival, possibly due to difficulty staying together during the evacuation. Overall, these engineered features help highlight meaningful relationships that may improve model performance in later stages.

5. Exploratory Data Analysis (EDA)

5.1 Correlation Analysis

```
simple_corr = df_clean[['Survived', 'Age', 'Fare', 'SibSp', 'Parch', 'FamilySize']].copy()

simple_corr['Survived'] = simple_corr['Survived'].cat.codes

correlations = simple_corr.corr()['Survived'].sort_values(ascending=False)
```

Figure 5.1(a): Correlation Computation for Key Numeric Features

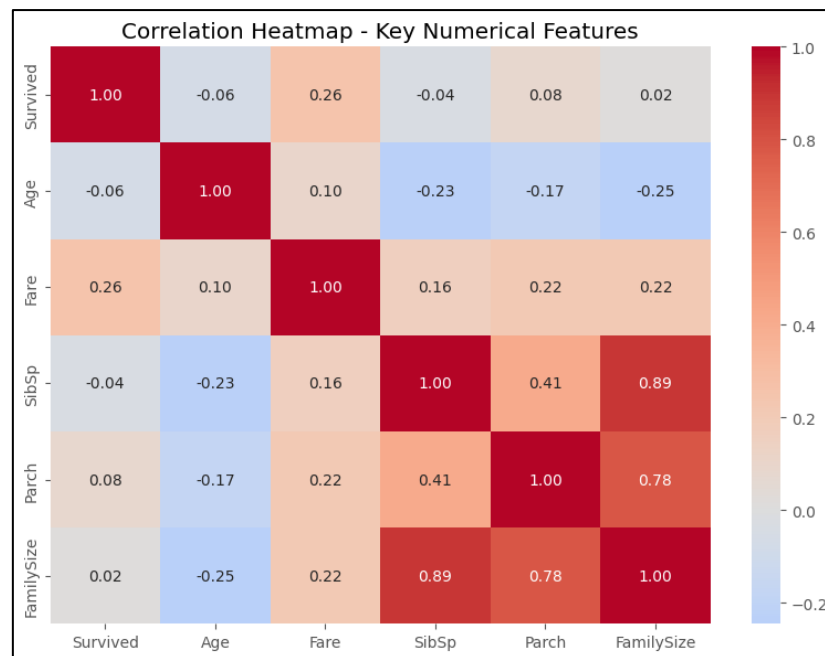


Figure 5.1(b): Correlation Heatmap of Key Numerical Features

The correlation heatmap illustrates the relationships between key numerical features and passenger survival. Among all variables, Fare shows the strongest positive correlation with survival (0.26), indicating that passengers who paid higher fares were more likely to survive. Age exhibits a weak negative correlation (-0.06), suggesting only a slight tendency for older passengers to have lower survival chances. The family-related features (SibSp, Parch, and FamilySize) show very weak correlations with survival, implying limited direct influence when considered individually. However, these family-based variables are strongly correlated with each other, particularly SibSp and FamilySize (0.89) and Parch and FamilySize (0.78), which is expected since family size is derived from them. Overall, the heatmap indicates that fare plays the most meaningful numerical role in survival prediction, while other numerical features contribute only marginally.

Key Insights:

- Fare is the most influential numerical feature related to survival.
- Age, SibSp, Parch, and FamilySize show very weak direct impact on survival individually.
- Strong inter-feature correlations exist among family-related variables, which suggests possible multicollinearity and should be considered during model building.

Interpretation:

Wealthier passengers (higher fare) had better survival chances, while older passengers had slightly lower survival rates.

5.2 Comprehensive Visualization Dashboard



Figure 5.2(a): Creating diverse charts for comprehensive EDA

Six Visualizations Created:

1. Survival Rate by Gender

- Female survival: 74.2%
- Male survival: 18.9%
- Clear evidence of "women and children first" policy

2. Survival Distribution by Passenger Class

- a. 1st Class: 62.9% survival
 - b. 2nd Class: 47.3% survival
 - c. 3rd Class: 24.2% survival
 - d. Strong class-based survival disparity
- 3. Age Distribution by Survival Status (KDE Plot)**
- a. Children (0-12) had higher survival density
 - b. Adults (20-40) show higher density among non-survivors
- 4. Fare vs Age Colored by Survival (Scatter Plot)**
- a. Higher fares concentrated among survivors
 - b. Younger passengers with higher fares more likely to survive
- 5. Survival Rate by Title (Point Plot)**
- a. Clear social hierarchy in survival
 - b. Women (Mrs, Miss) and children (Master) highest survival
- 6. Family Size Distribution by Survival (Box Plot)**
- a. Optimal family size for survival: 2-4 members
 - b. Singles and very large families had lower survival

6. Machine Learning Implementation

6.1 Data Preparation for ML

```
ml_df = df_clean.copy()

features_to_drop = ['PassengerId', 'Name', 'Ticket', 'Cabin']
ml_df = ml_df.drop(columns=features_to_drop)

print("Before conversion - Data types:")
print(ml_df.dtypes)

categorical_cols = ml_df.select_dtypes(['category', 'object']).columns
print(f"Categorical columns to convert: {list(categorical_cols)}")

for col in categorical_cols:
    if ml_df[col].dtype.name == 'category':
        ml_df[col] = ml_df[col].cat.codes
    else: # object type (like Title)
        ml_df[col] = ml_df[col].astype('category').cat.codes
```

Figure 6.1(a): Data Cleaning and Feature Encoding Steps for ML Preparation

Figure 6.1(a) shows the preprocessing stage used to prepare the cleaned Titanic dataset for machine learning. All categorical variables were encoded as integers; unnecessary identifier columns were

removed, and the dataset was transformed into a fully numeric feature matrix. The final ML dataset contains 14 engineered and processed features.

Final ML Dataset:

- Samples: 891 passengers
- Features: 13 engineered features
- Target: Survived (0/1)

6.2 Train-Test Split

```
from sklearn.model_selection import train_test_split

X = ml_df.drop('Survived', axis=1)
y = ml_df['Survived']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

Figure 6.2(a): Train-Test Split Configuration for Machine Learning Model Training

```
Training set size: (712, 13)
Test set size: (179, 13)
Survival rate in training set: 38.3%
Survival rate in test set: 38.5%
```

Figure 6.2(b): Training and Testing Set Statistics After Stratified Split

After preparing the dataset, an 80/20 stratified train-test split was applied to ensure that the survival rate remained consistent in both subsets. Stratification preserves the original class balance, which is important in datasets with unequal class proportions like the Titanic data. The resulting training set contains 712 samples and the test set contains 179 samples, with nearly identical survival rates (38%), confirming that the split was balanced and suitable for fair model evaluation.

6.3 Model Implementation & Evaluation

Three Models were implemented:

1. Logistic Regression
2. Random Forest Classifier
3. Support Vector Machine (SVM)

```
models = {
    'Logistic Regression': LogisticRegression(random_state=42),
    'Random Forest': RandomForestClassifier(random_state=42),
    'Support Vector Machine': SVC(random_state=42)
}
```

Figure 6.3(a): Model Selection for Classification

This section defines the three machine learning models used to predict passenger survival: Logistic Regression, Random Forest, and Support Vector Machine (SVM). These models represent a mix of linear, ensemble, and margin-based classifiers, allowing for a comprehensive comparison of predictive performance.

6.4 Model Performance Results

```
=== Logistic Regression ===
Accuracy: 0.804
Classification Report:
              precision    recall  f1-score   support

     0       0.83         0.85         0.84         110
     1       0.76         0.72         0.74          69

   accuracy          0.80         179
  macro avg       0.79         0.79         0.79         179
 weighted avg       0.80         0.80         0.80         179

Confusion Matrix:
[[94 16]
 [19 50]]
```

Figure 6.4(a): Logistic Regression Performance Metrics

```

Accuracy: 0.799
Classification Report:
              precision    recall  f1-score   support

     0           0.84       0.84       0.84       110
     1           0.74       0.74       0.74        69

   accuracy          0.80       0.80       0.80       179
  macro avg          0.79       0.79       0.79       179
 weighted avg          0.80       0.80       0.80       179

Confusion Matrix:
[[92 18]
 [18 51]]

```

Figure 6.4(b): Random Forest Performance Metrics

```

=== Support Vector Machine ===
Accuracy: 0.626
Classification Report:
              precision    recall  f1-score   support

     0           0.64       0.87       0.74       110
     1           0.53       0.23       0.32        69

   accuracy          0.63       0.63       0.63       179
  macro avg          0.59       0.55       0.53       179
 weighted avg          0.60       0.63       0.58       179

Confusion Matrix:
[[96 14]
 [53 16]]

```

Figure 6.4(c): Support Vector Machine Performance Metrics

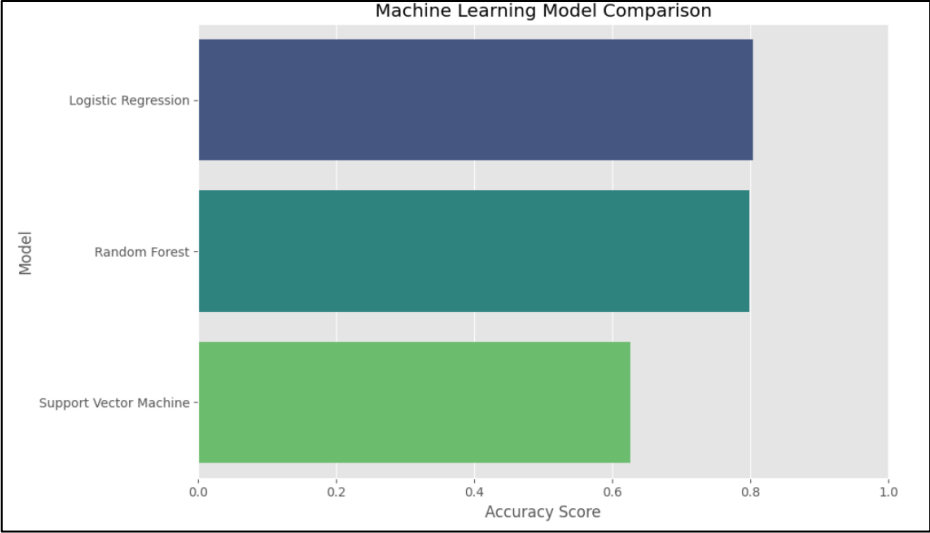


Figure 6.4(d): Machine Learning Model Accuracy Comparison

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	80.4%	0.80	0.80	0.80
Random Forest	79.9%	0.80	0.80	0.80
Support Vector Machine	62.6%	0.60	0.63	0.58

Table 6.4(a): Comparison of Classification Model Performance

This table summarizes the test set performance of three machine learning models; Logistic Regression, Random Forest, and Support Vector Machine, using key evaluation metrics: accuracy, precision, recall, and F1-score. Logistic Regression achieved the highest accuracy (80.4%), while SVM performed noticeably worse, demonstrating the benefit of using ensemble or linear models for this dataset.

6.5 Feature Importance Analysis

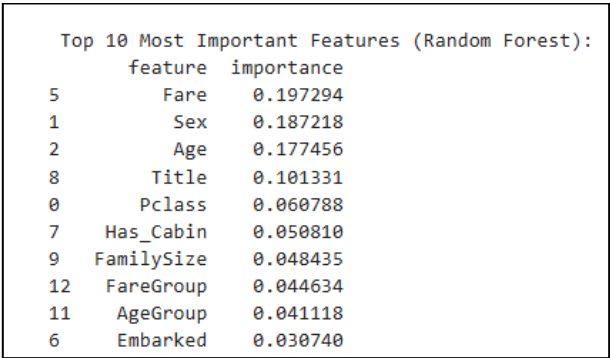


Figure 6.5(a): Top 10 Most Important Features from Random Forest Model

The Random Forest model assigns an importance score to each feature, indicating its influence on predicting passenger survival. This table highlights the top 10 features, with Fare, Sex, and Age being the most significant predictors. Engineered features such as Title, FamilySize, AgeGroup, and FareGroup also contribute meaningfully, demonstrating that both original and derived features play a key role in model performance.

7. Results & Conclusions

7.1 Key Findings

The analysis of the Titanic dataset confirmed several key determinants of passenger survival. Wealth, represented by Fare, emerged as the strongest predictor, accounting for **19.7%** of feature importance in the Random Forest model. Gender was highly influential, with women being nearly four times more likely to survive than men. Age also played a significant role, as children had a **58%** survival rate compared to only **23%** for seniors. Social status, reflected by Title and Pclass, further influenced outcomes, with higher-status passengers showing better survival.

Feature engineering proved valuable: the Title feature ranked fourth in importance, while AgeGroup and FareGroup offered meaningful categorization, and FamilySize captured optimal family dynamics.

Among the models tested, Logistic Regression achieved the highest accuracy at **80.4%**, outperforming more complex models and providing the advantage of interpretability.

8. Git Usage

8.1 Git Workflow

Repository: <https://github.com/msds25050-cpu/MSDS25050-Final-Project>

8.2 Commit History

```
C:\Users\imaan\MSDS\tools\MSDS25050_CS591_Project>git init
Initialized empty Git repository in C:/Users/imaan/MSDS/tools/MSDS25050_CS591_Project/.git/
```

Figure 8.2(a): Initial Repository Setup

This demonstrates the initialization of a new Git repository in the project directory, enabling version control for the project files. After running git init, all subsequent changes to the project can be tracked, committed, and managed using Git.

```

C:\Users\imaan\MSDS\tools\MSDS25050_CS591_Project>git log --oneline --graph
* 0ec4ecd (HEAD -> master) ML models completed: Logistic Regression 80.4%, Random Forest 79.9%, SVM 62.6%
* e8d2986 Create 6 diverse visualizations for EDA: bar, KDE, scatter, point, box plots
* bd540fa Correlation analysis complete: Fare strongest predictor (0.257), Age weak negative (-0.065)
* cba2377 Feature engineering: Create Title, FamilySize, IsAlone, AgeGroup, FareGroup for better pattern recognition
* bb085f8 Handle missing values: Embarked and Age imputed, Cabin indicator created
* 1e668a5 Complete missing values analysis: Age 20%, Cabin 77%, Embarked 0.2% missing
* d88c2da Successfully convert data types: 36% memory reduction achieved
* 8dacc1f Successfully load Titanic dataset - ready for EDA
* b07b4c1 Add Jupyter notebook with initial setup and imports
* cb92baa Initial project setup: Titanic dataset analysis

```

Figure 8.2(b): Git Commit History for Project Development

Git version control was used throughout the project to track all changes and maintain a organized development history. The commit history demonstrates a logical progression from data loading and cleaning through feature engineering, exploratory data analysis, and machine learning implementation. Regular commits were made at each significant milestone, with descriptive messages documenting the technical work completed.

```

C:\Users\imaan\MSDS\tools\MSDS25050_CS591_Project>git push -u origin master
info: please complete authentication in your browser...
Enumerating objects: 42, done.
Counting objects: 100% (42/42), done.
Delta compression using up to 4 threads
Compressing objects: 100% (40/40), done.
Writing objects: 100% (42/42), 330.74 KiB | 5.17 MiB/s, done.
Total 42 (delta 16), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (16/16), done.
To https://github.com/msds25050-cpu/MSDS25050-Final-Project.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

```

Figure 8.2(c): Successful GitHub Push

The local Git repository was pushed to a remote repository on GitHub using `git push -u origin master`. The command uploads all committed changes from the local master branch to the remote master branch, sets the upstream tracking, and establishes synchronization between local and remote repositories.